



Python Programming – Week 10: Files

Post-class Exercises & Challenges

For each exercise below:

- ✓ Write **one or more functions**
- ✓ Call them from the **main program**
- ✓ Read from file(s) and write output file(s)

Exercise 1 — Trim and Re-save

Input: `raw.txt`

Task: Create `trimmed.txt` where each line is stripped of leading/trailing spaces.

Requirements:

1. Keep the same number of lines as `raw.txt`
 2. Do not remove empty lines; an empty line remains empty
-

Exercise 2 — Extract Email-like Lines

Input: `contacts.txt` (one contact per line)

Task: Create `emails_only.txt` containing only lines that contain the character @.

Requirement: Preserve original line order.

Exercise 3 — Replace Words from a Mapping

Inputs: `text.txt`, `mapping.txt`

- `mapping.txt` format: each line is `old_word, new_word` (one pair per line)

Task: Create `replaced.txt` by replacing every occurrence of `old_word` with `new_word` for all pairs in `mapping.txt`.

Requirement: Apply mappings in the order they appear in `mapping.txt`.

Exercise 4 — Find Top 3 Longest Lines

Input: `input.txt`

Task: Create `top3_longest.txt` containing the **3 longest lines** from the file (by character count).

Requirements:

1. Keep the original text of those lines
 2. If ties occur, keep the earlier line first
 3. Output exactly 3 lines (assume input has at least 3 lines)
-

Exercise 5 — Running Total (Numbers File)

Input: `numbers.txt` (one integer per line)

Task: Create `running_total.txt` where each line is the cumulative sum up to that line. Example idea: if input is 2, 5, -1 then output lines are 2, 7, 6.

Exercise 6 — CSV: Select + Rename Columns

Input: `students.csv` with columns `name, age, major, gpa`

Task: Create `students_export.csv` containing only `name, gpa` but with headers renamed to:

`student_name, final_gpa`

Requirement: Keep row order unchanged.

Exercise 7 — CSV: Filter by Exact Match

Input: `students.csv` with columns `name, age, major, gpa`

Task: Ask the user for a major (e.g., `CS`). Create `major_filtered.csv` containing only students whose `major` matches exactly.

Requirement: Keep the header row.

Exercise 8 — CSV: Count Per Category

Input: `transactions.csv` with columns `date, category, amount`

Task: Create `category_counts.txt` showing how many rows belong to each category.

Requirements:

1. Output format: `category: count` (one per line)
2. Categories should be listed in alphabetical order

Exercise 9 — CSV: Find Maximum Amount Row

Input: `transactions.csv` with columns `date`, `category`, `amount`

Task: Create `max_transaction.txt` containing exactly one line describing the row with the largest `amount`.

Requirements:

1. Output format: `date=<...>, category=<...>, amount=<...>`
 2. If there is a tie, choose the earliest row in the file
-

Exercise 10 — CSV: Add a Status Column

Input: `scores.csv` with columns `student`, `score`

Task: Create `scores_with_status.csv` with columns `student`, `score`, `status` where:

- `status` = `PASS` if `score` $\geq$ 50
- `status` = `FAIL` otherwise

Requirement: Keep the header row and original row order.